Enumerated Types

Lecture 10

Based on Slides of Dr. Norazah Yusof

Enumerated Types

- Enumeration is a special kind of class, that is introduced by the keyword enum and enum constants.
- Syntax: `enum` *`typeName`* `{`*one or more enum constants*`}`
- Example : enumeration definition:

```
enum Day {SUNDAY, MONDAY, TUESDAY,
WEDNESDAY, THURSDAY, FRIDAY, SATURDAY}
```

- Declaration: `Day workday;`
- Operation: `workday = Day.WEDNESDAY;`

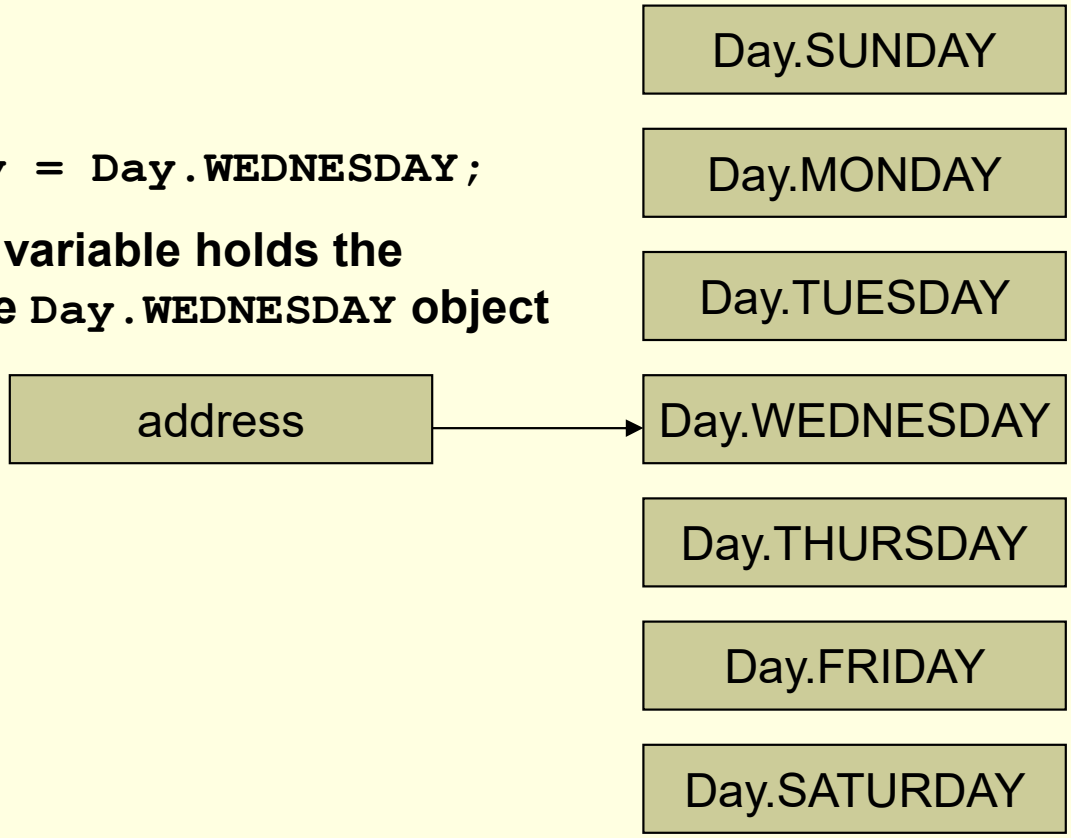
Enumerated Types

- **An `enum` is a specialized class**

Each are objects of type `Day`, a specialized class

```
Day workDay = Day.WEDNESDAY;
```

The `workDay` variable holds the address of the `Day.WEDNESDAY` object



Enumerated Types restriction

- `enum` types are implicitly `final`, because they declare constants that should not be modified
- `enum` constants are implicitly `static`
- Any attempt to create an object of an enum type with operator `new` results in a compilation error

enum Example

```
public class EnumTest1{
  enum Day {SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
  THURSDAY, FRIDAY, SATURDAY}
  public static void main(String[] args){
   Day workday;
   workday = Day.THURSDAY;
   System.out.println ("Today is "+ workday);
   System.out.println ("Yesterday was " +
     Day.WEDNESDAY);
 }
}
```

enum methods

- enum class is equipped with a few methods:
- `ordinal()` method returns an integer value representing the constant's ordinal value.
- `equals()` method accepts an object as its argument and returns true if that object is equal to the calling enum constant.
- `compareTo()` method accepts an object as its argument and returns negative integer, zero, or positive value if the calling enum constant's ordinal value is less than, equal or greater than the argument's ordinal value.

enum methods

- Example 1:

```
System.out.println ("Ordinal value: “ +
workday.ordinal());
```

- Example 2:

```
Day myDay = Day.FRIDAY;
if (myDay.equals(workday))
   System.out.println ("same day ");
else
   System.out.println ("different day ");
```

enum with multiple values (opt1)

```
public enum typeName {
    const_1(val_1, . . ., val_n),
    . . .
    const_n(val_1, . . ., val_n);

    public final type1 var_1;
    . . .
    public final type1 var_1;


    private typeName(type1 v_1, . . ., type2 v_n) {
        var_1 = v_1;
        . . .
        var_n = v_n;
    }

    public type methodName() {

    }
}
```

enum with multiple values (opt2)

```
public enum typeName {
    const_1(val_1, . . ., val_n),
    . . .
    const_n(val_1, . . ., val_n);

    private type1 var_1;
    . . .
    private type1 var_1;


    private typeName(type1 v_1, . . ., type2 v_n) {
        var_1 = v_1;
        . . .
        var_n = v_n;
    }

    public type getVar_1() {
         return var_1;
    }
}
```

enum with multiple values

- How to use enum to model the following plane type information?

Code	Type	Speed
B	Bomber	600
F	Fighter	1600

Plane enum (opt1)

Plane.java

```
----------
public enum Plane {
    B("Bomber", 800),
    F("Fighter", 1600);

    public final String type;
    public final int speed;

    private Plane(String t, int s) {
        type = t;
        speed = s;
    }
}
```

Plane enum (opt2)

Plane.java

```
----------
public enum Plane {
    B("Bomber", 800),
    F("Fighter", 1600);

    private String type;
    private int speed;

    private Plane(String t, int s) {
        type = t;
        speed = s;
    }

    public String getType() { return type; }
    public int getSpeed() { return speed; }
}
```

Plane enum

- Iterate Plane enum data type (opt1)

```
for (Plane p : Plane.values()) {
    System.out.println(p + " " + p.type + " " + p.speed);
}
```

- Iterate Plane enum data type (opt2)

```
for (Plane p : Plane.values()) {
    System.out.println(p + " " + p.getType() + " " +
                       p.getSpeed());
}
```

Plane enum

- Approaches to assign Plane enum type data

```
Plane f16 = Enum.valueOf(Plane.class, "F");
Plane b52 = Plane.B; // can imagine it like:
                     // Plane b52 = new Plane("Bomber", 600);
```

- Add compareSpeed method so the following statement return 1000 (1600 – 600)

```
System.out.print("f16.compareSpeed(b52): " +
                 f16.compareSpeed(b52));
```

Lab 4

- Exercises on Enum types
  - Do Exercise 2, Question 5 on page 88
  - Do Exercise 2, Question 6 on page 89

Java Package

Lecture 11

Based on Slides of Dr. Norazah Yusof

Putting Classes into Packages

- Every class in Java belongs to a package.
- The class is added to the package when it is compiled.
- All the classes that you have used so far were placed in the active directory (a default package) when the Java source programs were compiled.
- To put a class in a specific package, you need to add the following line as the first non-comment and non-blank statement in the program: `package package_name;`

Why we need package?

There are three reasons for using packages:

1. *To avoid naming conflicts*. When you develop reusable classes to be shared by other programmers, naming conflicts often occur. To prevent this, put your classes into packages so that they can be referenced through package names.
2. *To distribute software conveniently*. Packages group related classes so that they can be easily distributed.
3. *To protect classes*. Packages provide protection so that the protected members of the classes are accessible to the classes in the same package, but not to the external classes.

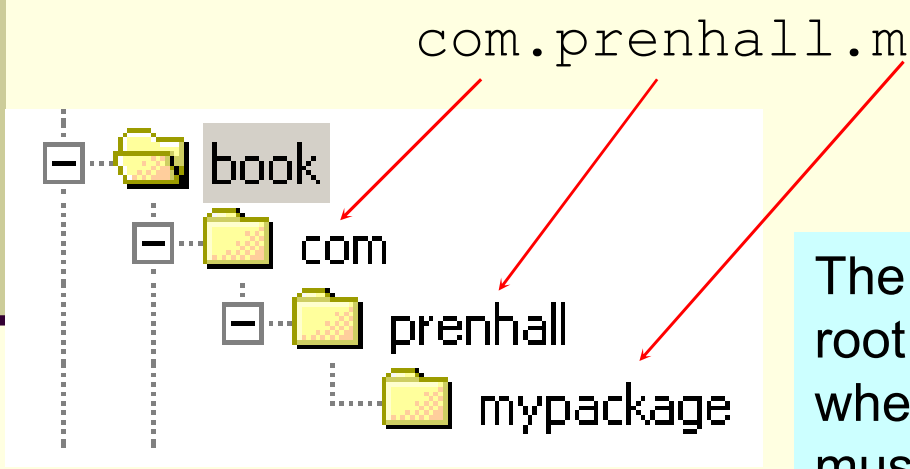
Package-Naming Conventions

- Packages are hierarchical.
- Can have packages within packages.
- For example:
  - java.lang.Math indicates that Math is a class in the package lang and that lang is a package in the package java.
- Levels of nesting can be used to ensure the uniqueness of package names.
- Choosing a unique name for your package is important for the sake of reusability.
- By convention, package names are all in lowercase

Package Directories

Java expects one-to-one mapping of the package name and the file system directory structure.

For the package named `com.prenhall.mypackage`, you must create a directory, as shown in the figure. In other words, a package is actually a directory that contains the bytecode of the classes.



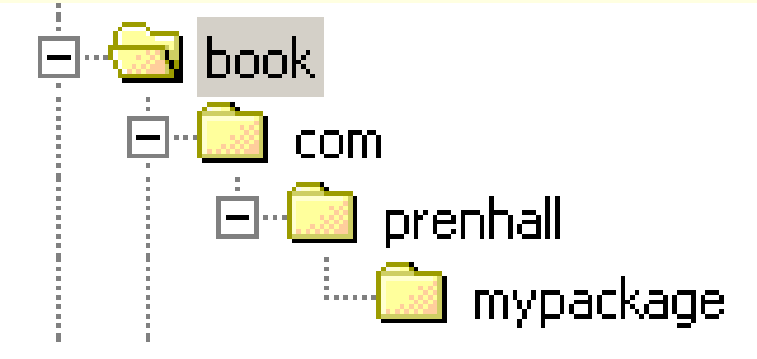
The com directory does not have to be the root directory. In order for Java to know where your package is in the file system, you must modify the environment variable classpath so that it points to the directory in which your package resides.

Setting classpath Environment

Suppose the com directory is under c:\book. The following line adds c:\book into the classpath:

```
classpath=.;c:\book;
```

The period (.) indicating the current directory is always in classpath. The directory c:\book is in classpath so that you can use the package com.prenhall.mypackage in the program.



Setting classpath Environment in different platform

- Windows 98: Edit `autoexec.bat` using a text editor to set the `classpath`.
- Windows NT/2000/XP: Modify the `classpath` in the Environment Variables
- UNIX: Use the `setenv` command to set `classpath`.

Exercise: Putting Classes into Packages

Problem (Exercise 1, page 80-85)

1.Create three classes named Bulatan , SegiEmpat, dan SegiTigaTepat and place it in the folder `bentuk` that is in another folder `mypackage` from the root (or package `mypackage.bentuk`).

2.Create a test program named `Contoh3.java` that uses the classes. The test program is placed in different folder named `prog3` and in the root directory.

Solution

i.Create directory named `mypackage` from the root direcotry.

ii.Create directory named `bentuk` under the directory mypackage.

iii.Create Bulatan.java and save it into c:\mypackage\bentuk. The Bulatan class contains the luas() and the paparMaklumat() methods.

```
// Bulatan.java
        package bentuk;
        public class Bulatan {
           private int titikTengahX, titikTengahY;
           private int jejari;
           :
           :   }
```

Exercise: Putting Classes into Packages

Solution continue…

v.Compile Bulatan.java. Make sure Bulatan.java is in c:\mypackage\bentuk

vi.Create Segiempat.java as well as SegiTigaTepat.java and save it into `c:\mypackage\bentuk`.

vii.Compile Segiempat.java and SegiTigaTepat.java.

Exercise: Using Packages

Problem

This example shows a program created in directory `prog3`, that uses the Bulatan, Segiempat dan SegiTigaTepat class in the `mypackage.bentuk` package.

Solution

1. Create Contoh3.java as follows and save it into c:\prog3.

The following code gives the solution to the problem.

```
// Contoh3.java: Demonstrate using the Bulatan class
  import bentuk.*;
    public class Contoh3 {
      public static void main(String[] args) {
        Bulatan a = new Bulatan(0, 0, 10);
        System.out.println("Maklumat bentuk a");
        a.paparMaklumat();
        System.out.println("Luas:" + a.luas());
      }
    }
```

Exercise: Set Environment

Solution

Set the path in a batch file.

```
set PATH=C:\Program Files\Java\jdk1.5.0_01\bin
set CLASSPATH=.;c:\mypackage;
cd \prog3
```

Or

Set the path from the computer environment.

Using Classes from Packages

There are two ways to use classes from a package.

1. One way is to use the fully qualified name of the class.

```
import bentuk.Bulatan;
```

2. The other way is to use the import statement. For example, to import all the classes in the javax.swing package, you can use

```
import bentuk.*;
```

- An import that uses a * is called an import on demand declaration. You can also import a specific class. For example:

```
import bentuk.Bulatan;
```

- The information for the classes in an imported package is not read in at compile time or runtime unless the class is used in the program. The import statement simply tells the compiler where to locate the classes.

Lab 4

- Do Exercises